CDN and HTTP Caching

the headers that decide who may cache what, for how long

Phase 3 videos on YouTube

In one line: Cache-Control decides caching: max-age for everyone, s-maxage for shared caches like CDNs, public or private (only the user's browser), no-store for sensitive data; ETags with If-None-Match turn re-downloads into cheap 304s, Vary and the CDN cache key decide what counts as the same response, and surrogate keys purge related responses at once.

1 Two problems, one cause

Your API's product list is the same for every visitor, yet every request travels to your origin servers across the world. Meanwhile, a user's private account page got cached at the CDN and shown to someone else. Both problems come down to one thing: **HTTP caching headers**. (Phase 2, lessons 19 and 28 introduced status codes, ETags and CDNs; here we go deeper.)

2 Cache-Control

- **max-age=N**: any cache may reuse the response for N seconds.
- **s-maxage=N**: overrides max-age for **shared** caches (CDNs, proxies), so the CDN can keep it longer than browsers.
- **public**: shared caches may store it (even with an Authorization header).
- **private**: only the user's own browser may cache it, never a shared cache.
- **no-cache**: may be stored, but must be revalidated with the server before each use.
- **no-store**: don't store it anywhere.

`public, max-age=60`	browser and CDN may keep it 60 s
`public, max-age=10, s-maxage=300`	browser 10 s, CDN / proxies 300 s
`private, max-age=60`	only the user's browser; never a CDN
`no-cache`	may store, but must revalidate before every use
`no-store`	don't store anywhere: banking, health pages

Figure 1. Common Cache-Control values and what each one allows.

What does private mean? It's for responses specific to one user, like an account page or a cart. The browser may cache it, but a CDN or corporate proxy must not, because it could serve one user's data to another. For anything sensitive, like banking pages, use **no-store** instead. And never let a CDN cache a response that depends on a cookie unless the cookie is part of the cache key.

3 Validation: ETags and 304

When a cached response expires, it doesn't have to be downloaded again. The server sends an **ETag**, a fingerprint of the content (a hash or a version). Later the client sends `If-None-Match` with that ETag. If

nothing changed, the server replies **304 Not Modified** with no body, and the client keeps its copy. `Last-Modified` and `If-Modified-Since` work the same way with timestamps.

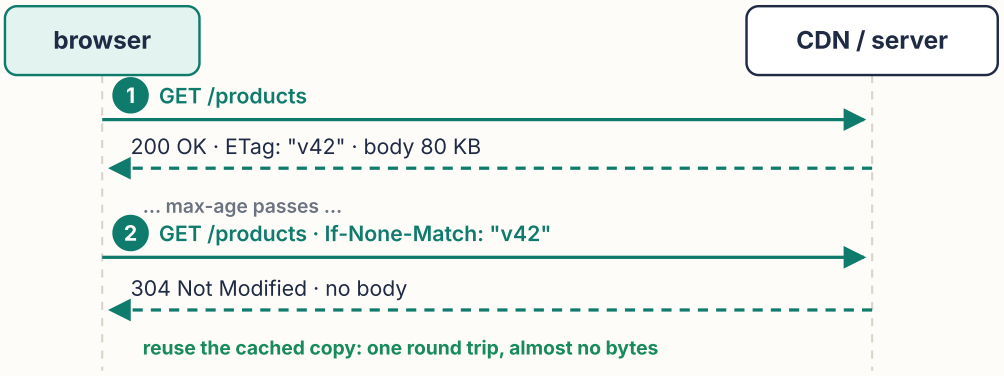


Figure 2. Revalidation with an ETag: an unchanged resource costs one small round trip instead of a full download.

4 Vary and stale-while-revalidate

Vary tells caches which request headers change the response, like `Accept-Encoding` or `Accept-Language`, so a gzip version isn't served to a client that can't read it. Use it carefully: `Vary: Cookie` or `Vary: User-Agent` explodes the number of cache variants and the hit ratio collapses. **stale-while-revalidate=N** lets a cache serve a slightly stale copy for N seconds while it fetches a fresh one in the background (lesson 24).

5 At the CDN: cache keys and purges

The CDN's **cache key** decides what counts as the same response: usually host, path and query string. Strip tracking parameters (`utm_*`, `gclid`) and sort the rest, or every marketing link becomes its own miss. **Surrogate keys** (cache tags) let you purge every response related to one product with a single API call, instead of guessing URLs (lesson 22).

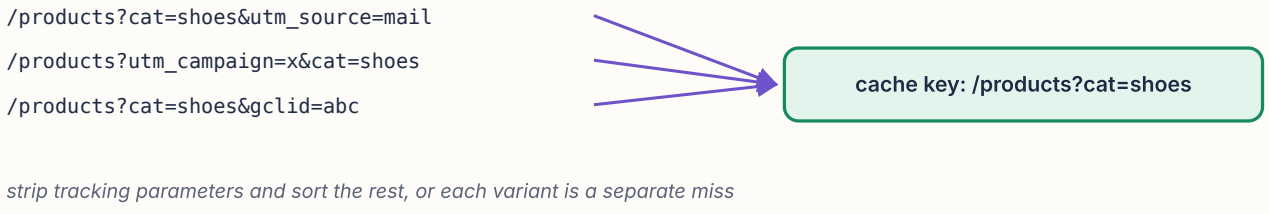


Figure 3. Normalising the cache key: three URLs, one cached response.

WORKED EXAMPLE: GET /PRODUCTS

A public endpoint returns the same list for everyone. It sends `Cache-Control: public, max-age=10, s-maxage=30` plus an ETag. The CDN serves it for 30 seconds from the edge, so the origin sees about **one request per edge location every 30 seconds**, instead of thousands. Browsers revalidate after 10 seconds and usually get a cheap 304.

COMMON MISTAKES

- `public` (or no header, with a CDN that caches by default) on user-specific pages: data leaks between users.
- Long max-age on HTML or API responses you'll need to change: you can't purge browsers. Use long max-age only for versioned assets (`app.3f9a.js`).
- `Vary: Cookie` on a CDN: almost zero hit ratio.
- Confusing no-cache (revalidate) with no-store (never store).

6 Key terms

max-age / s-maxage: freshness for all caches / for shared caches

no-store: never store the response

ETag: content fingerprint for revalidation

Vary: request headers that change the response

public / private: shared caches allowed / only the browser

no-cache: store, but revalidate before use

304 Not Modified: unchanged: reuse your copy

Cache key: what the CDN treats as the same response

7 Interview questions

Q1. What does Cache-Control: private mean? BASIC

Only the end user's browser may cache the response; shared caches like CDNs and proxies must not, because it's user-specific. Use no-store for sensitive data.

Q2. How do ETags work? BASIC

The server sends a content fingerprint; the client later sends If-None-Match with it, and if nothing changed the server replies 304 Not Modified without a body, so the cached copy is reused.

Q3. max-age vs s-maxage? BASIC

max-age applies to every cache; s-maxage overrides it for shared caches, so you can let a CDN keep a response longer than browsers, where you can't purge.

Q4. Cache a public product list at the CDN without stale prices for minutes. SCENARIO

Short s-maxage (e.g. 30 s) plus stale-while-revalidate, an ETag, surrogate-key purges on price changes, and a short browser max-age.

Q5. A user saw another user's account page. What happened and how do you fix it? SCENARIO

The response was cached by a shared cache: it lacked private/no-store, or the CDN ignored the session cookie. Send Cache-Control: private or no-store on personal pages, and configure the CDN to bypass cache when a session cookie is present.

Q6. Why can Vary destroy your CDN hit ratio? TRICKY

Each distinct value of a varied header becomes a separate cached variant; varying on Cookie or User-Agent creates nearly one variant per user or browser. Vary only on low-cardinality headers like Accept-Encoding.

Next: SD-F-018 · Multi-Level Caching backendroadmap.com